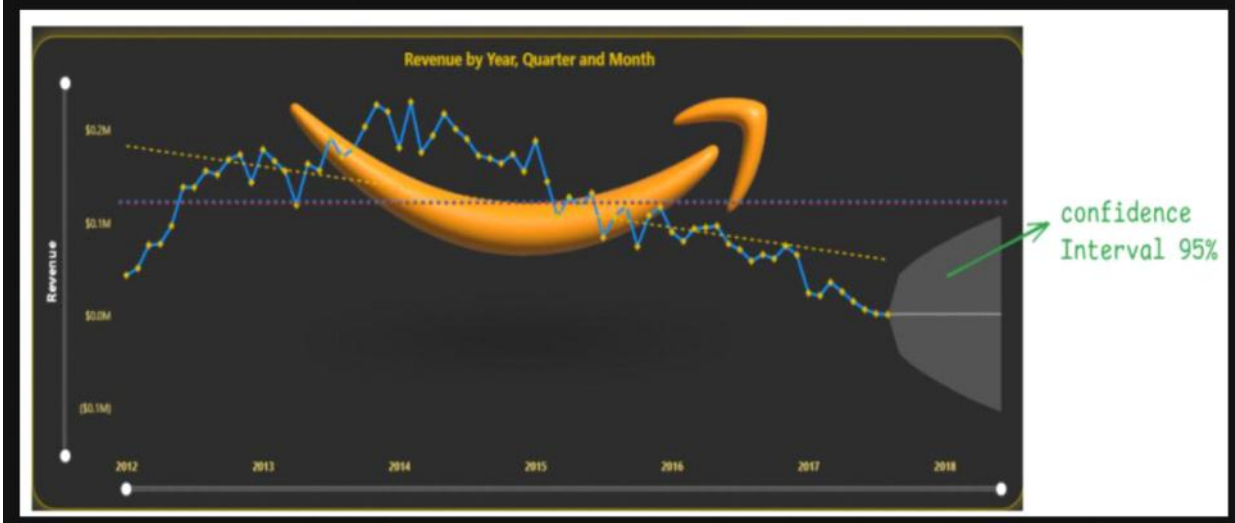
Confidence Interval & Statistical Test

🎯 Session Objectives:

- ✅ Understand the concept of confidence intervals and how they relate to hypothesis testing.
- ✅ Construct confidence intervals using Python with detailed examples.
- ✅ Gain a basic understanding of Type I and Type II errors, relating them to the concepts of false positives and false negatives.
- ✅ Understand the concept and purpose of the T-Test in statistical analysis.
- ✅ Calculate and interpret the Standard Error and its significance in hypothesis testing.
- ✅ Explain the role of Degrees of Freedom and how they affect statistical distributions.
- ✅ Distinguish between T-Distribution and Normal Distribution and know when to apply each.
- ✅ Perform and interpret One-Sample T-Tests, ANOVA, and Chi-Square Tests using Python.



Impact of Sample Size on Confidence Intervals

Larger Sample Size

When sample size increases, the standard error decreases. A smaller standard error results in a **narrower confidence interval**, providing a more precise estimate of the population parameter.

Example: A sample of 100 flour packages yields a narrower 95% CI compared to 25 packages, because the sample mean is more reliable.

Smaller Sample Size

A smaller sample size increases the standard error, leading to a **wider confidence interval**. This means our estimate is less precise, and the range is broader.

Example: Sampling only 10 flour packages produces a wider 95% CI due to greater uncertainty from the smaller sample.

```python
# Confidence Interval :
import numpy as np

# Given Data:
population_mean = 1.0 # in kg
sample_mean = 0.98 # in kg,
sample_size = 40,
std = 0.1 # in kg,
confidence_level = 0.95

standard_error = std / np.sqrt(sample_size)
print(standard_error)
```

```
[0.01581139]
```

```python
# For 95% Confidence Level -> z_score = 1.96
z_score = 1.96

# Confidence Interval
confidence_interval = (
    sample_mean - (z_score * standard_error),
    sample_mean + (z_score * standard_error)
)
confidence_interval
```

```
(array([0.94900968]), array([1.01099032]))
```

1

Sample Proportion Formula

$\hat{p} = x/n$ where x is number of successes and n is total observations

2

Mean of Sampling Distribution

Mean of $\hat{p} = \mu_{\hat{p}} = p$ (equals the population proportion)

3

Standard Error of Proportion

$SE = \sqrt{[p(1-p)/n]}$ measures variability of sample proportion

```python
# For 95% Confidence Level -> z_score = 1.96
z_score = 1.96

# Confidence Interval
confidence_interval = (
    sample_proportion - (z_score * standard_error),
    sample_proportion + (z_score * standard_error)
)
confidence_interval
```

```
(0.0503372315640978, 0.1296627684359022)
```

```
We are 95% confident that the true proportion of defective bulbs in the batch lies between 5.1% and 12.9%.
```

```python
# Sample Size and Statistical Power
import numpy as np
from scipy.stats import norm

def calculate_power(sample_size, effect_size , std_dev , alpha = 0.05):
    standard_error = std_dev / np.sqrt(sample_size)
    z_critical = norm.ppf(1-alpha) # 95%  CL
    z_effect = effect_size / standard_error
    power = 1 - norm.cdf(z_critical - z_effect)
    return power

effect_size = 5
std_dev = 10
alpha = 0.05

# Testing Different Sample Sizes
sample_size = [20,50,100,200,500]

for size in sample_size:
    power = calculate_power(size, effect_size, std_dev, alpha)
    print(f"Sample Size {size} : Power : {power:.2f}")
```

```
Sample Size 20 : Power : 0.72
Sample Size 50 : Power : 0.97
Sample Size 100 : Power : 1.00
Sample Size 200 : Power : 1.00
Sample Size 500 : Power : 1.00
```

Sample Size and Statistical Power

Example: Testing a Blood Pressure Drug

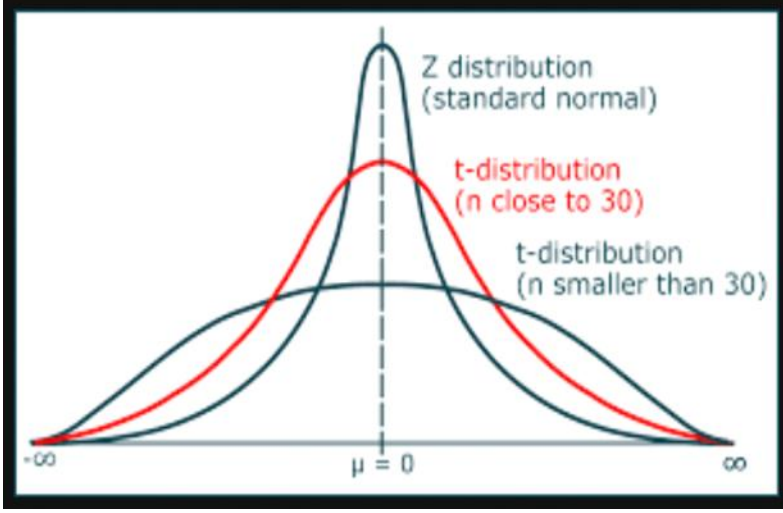
We're testing whether a new drug lowers blood pressure compared to placebo. The drug reduces blood pressure by 5 units (true effect), with population standard deviation of 10 units, and $\alpha = 0.05$.

1. **Sample Size: 20**
   Power: 0.36 (36% chance of detecting the effect)
2. **Sample Size: 50**
   Power: 0.61 (61% chance of detecting the effect)
3. **Sample Size: 100**
   Power: 0.82 (82% chance of detecting the effect)
4. **Sample Size: 200**
   Power: 0.95 (95% chance of detecting the effect)
5. **Sample Size: 500**
   Power: 0.99 (99% chance of detecting the effect)

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

χ^2 = the test statistic $\sum$ = the sum of

O = Observed frequencies E = Expected frequencies



https://www.sjsu.edu/faculty/gerstman/StatPrimer/t-table.pdf T-Test

https://www.stat.purdue.edu/~lfindsen/stat503/F_alpha_05.pdf F-Test[Anova Test]

https://www.stat.purdue.edu/~lfindsen/stat511/F_alpha_01.pdf F-Test[Anova Test]

https://math.arizona.edu/~jwatkins/chi-square-table.pdf Chi-Square Test X^2

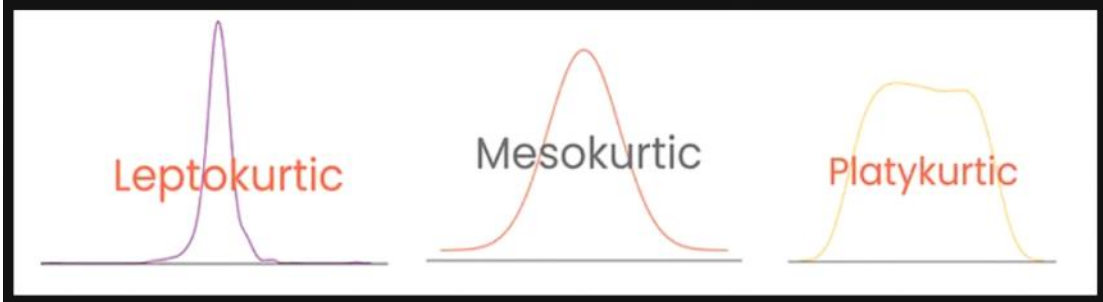
Degree of Freedom -> Within [k-1]
Degree of Freedom -> Between [n-k]

Use a **z-score** when the **population standard deviation is known** and the **sample size is large (n ≥ 30)**, or when the population is normally distributed with known parameters. Use a **t-score** when the **population standard deviation is unknown** and must be estimated from the **sample standard deviation**, particularly when the **sample size is small (n < 30)**.

Confidence Level	Z-Score (Two-tailed)	% in Each Tail
80%	1.28	10%
85%	1.44	7.5%
90%	1.645	5%
92%	1.75	4%
95%	1.96	2.5%
96%	2.05	2%
97%	2.17	1.5%
98%	2.33	1%
99%	2.575	0.5%
99.5%	2.807	0.25%
99.9%	3.291	0.05%

Small Table of z-values for Confidence Intervals

Confidence Level	z
0.70	1.04
0.75	1.15
0.80	1.28
0.85	1.44
0.90	1.645
0.92	1.75
0.95	1.96
0.96	2.05
0.98	2.33
0.99	2.58



t Table

2.5% - 1%

cum. prob	$t_{.50}$	$t_{.75}$	$t_{.80}$	$t_{.85}$	$t_{.90}$	$t_{.95}$	$t_{.975}$	$t_{.99}$
one-tail	0.50	0.25	0.20	0.15	0.10	0.05	0.025	0.01
two-tails	1.00	0.50	0.40	0.30	0.20	0.10	0.05	0.02
df								
1	0.000	1.000	1.376	1.963	3.078	6.314	12.71	31.82
2	0.000	0.816	1.061	1.386	1.886	2.920	4.303	6.965
3	0.000	0.765	0.978	1.250	1.638	2.353	3.182	4.541
4	0.000	0.741	0.941	1.190	1.533	2.132	2.776	3.747
5	0.000	0.727	0.920	1.156	1.476	2.015	2.571	3.365
6	0.000	0.718	0.906	1.134	1.440	1.943	2.447	3.143
7	0.000	0.711	0.896	1.119	1.415	1.895	2.365	2.998
8	0.000	0.706	0.889	1.108	1.397	1.860	2.306	2.896
9	0.000	0.703	0.883	1.100	1.383	1.833	2.262	2.821
10	0.000	0.700	0.879	1.093	1.372	1.812	2.228	2.764
11	0.000	0.697	0.876	1.088	1.363	1.796	2.201	2.718
12	0.000	0.695	0.873	1.083	1.356	1.782	2.179	2.681
13	0.000	0.694	0.870	1.079	1.350	1.771	2.160	2.650
14	0.000	0.692	0.868	1.076	1.345	1.761	2.145	2.624
15	0.000	0.691	0.866	1.074	1.341	1.753	2.131	2.602

2.31

t_stats

alpha = 0.05 ~ 5%

```
t_stats = (sample_mean - population_mean)/standard_error
t_stats
```

```
2.309401076758503
```

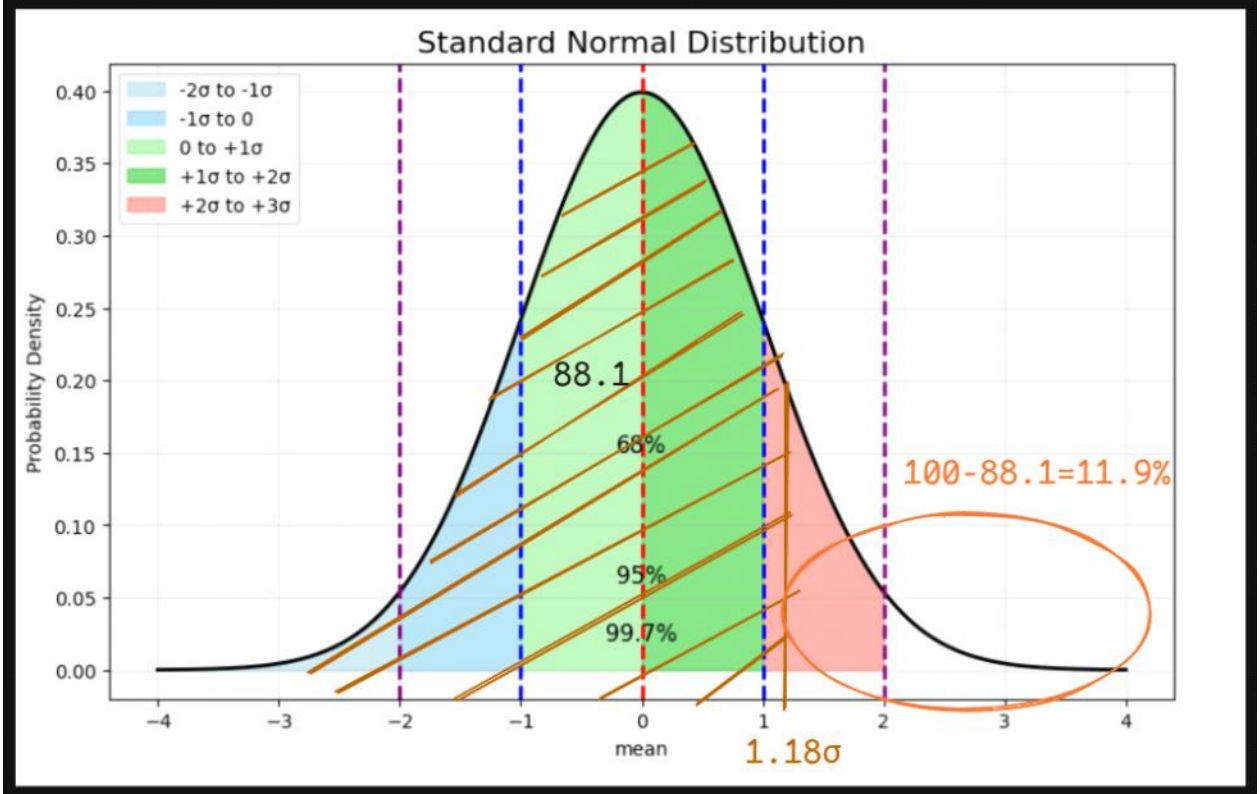
```
2.309401076758503
```

```
degree_of_freedom = sample_size - 1 # df = n - 1
degree_of_freedom
```

```
11
```

```
# p-value -> One-Sample T-Test
p_value = stats.t.sf(t_stats , degree_of_freedom)
p_value
```

```
0.020671133685861775
```
2.067%




```
# One-Sample T-Test: Company Sales Analysis

import numpy as np
import scipy.stats as stats

sample_mean = 210000
population_mean = 200000
std = 15000 # Sample Std(s) as pop.std is unknown
sample_size = 12 # n<=30 : T-Test

standard_error = std/np.sqrt(sample_size)
standard_error
```

```
4330.127018922193
```

```
t_stats = (sample_mean - population_mean)/standard_error
t_stats
```

```
2.309401076758503
```

```
degree_of_freedom = sample_size - 1 # df = n - 1
degree_of_freedom
```

```
11
```

```
# p-value -> One-Sample T-Test
p_value = stats.t.sf(t_stats , degree_of_freedom)
p_value
```

```
0.020671133685861775
```

```
# One-Sample T-Test : Diet Effectiveness Study
import numpy as np
import scipy.stats as stats

sample_mean = 210 # mg/dL
population_mean = 220 # mg/dL
std = 12 # Sample Std(s) as pop.std is unknown
sample_size = 15 # n<=30 : T-Test

standard_error = std / np.sqrt(sample_size)
standard_error
```

```
3.0983866769659336
```

```
t_stats = (sample_mean - population_mean)/standard_error
t_stats
```

```
-3.227486121839514
```

```
degree_of_freedom = sample_size - 1 # df = n - 1
degree_of_freedom
```

```
14
```

```
# p-value -> One-Sample T-Test
p_value = stats.t.sf(t_stats , degree_of_freedom)
p_value
```

```
0.9969605224321539
```